8장 정답 (Answers)

8.10.1 단답형

1. mov esp,ebp → pop ebp → ret n (n = 매개변수 바이트 수, C 방식은 그냥 ret)

2. EAX

3. 호출된 프로시저가 RET n 으로 매개변수 n바이트를 제거한다.

4. LEA는 실행 중에 계산한 주소(스택 지역 변수 [ebp-n], 간접 주소 등)를 구할 수 있고, OFFSET은 어셈블 시점에 정해진 정적 주소만 가능하다.

5. 4바이트

6. 호출자가 스택을 정리하므로 가변 개수 인자를 받는 함수(printf 등)를 만들 수 있다.

7. 거짓 (쉼표 뒤에서 줄을 바꿔 여러 줄에 쓸 수 있음)

8. 거짓 (포인터는 모두 32비트 주소라 어셈블러가 대상 타입을 검사하지 않음)

9. 참

8.10.2 알고리즘 워크벤치

1. push ebp 직후의 스택 프레임 (높은 주소 → 낮은 주소)

```
10h             [ESP+16]   처음 push
20h             [ESP+12]
30h             [ESP+8]    마지막 push
반환 주소        [ESP+4]
저장된 EBP       [ESP]  ← ESP
```

2.

```
AddThree PROC, val1:DWORD, val2:DWORD, val3:DWORD
    mov  eax,val1
    add  eax,val2
    add  eax,val3
    ret
AddThree ENDP
```

3 ~ 7.

```
LOCAL pArray:PTR DWORD        ; 3
LOCAL buffer[20]:BYTE         ; 4
LOCAL pwArray:PTR WORD        ; 5
LOCAL myByte:SBYTE            ; 6
LOCAL myArray[20]:DWORD       ; 7
```

8.

```
SetColor PROC, forecolor:DWORD, backcolor:DWORD
    mov  eax,backcolor
    shl  eax,4
    or   eax,forecolor
    call SetTextColor
    ret
SetColor ENDP
```

9.

```
WriteColorChar PROC USES eax, char:DWORD, forecolor:DWORD, backcolor:DWORD
    mov  eax,backcolor
    shl  eax,4
    or   eax,forecolor
    call SetTextColor
    mov  eax,char
    call WriteChar
    ret
WriteColorChar ENDP
```

10.

```
DumpMemory PROC USES ebx ecx esi, ptrArray:PTR BYTE, itemCount:DWORD, itemSize:DWORD
    mov  esi,ptrArray
    mov  ecx,itemCount
    mov  ebx,itemSize
    call DumpMem
    ret
DumpMemory ENDP
```

11.

```
MultArray PROTO, ptr1:PTR DWORD, ptr2:PTR DWORD, count:DWORD

MultArray PROC, ptr1:PTR DWORD, ptr2:PTR DWORD, count:DWORD
    ret
MultArray ENDP
```

8.11 프로그래밍 연습 문제 정답

★ 1. FindLargest

```
INCLUDE Irvine32.inc

FindLargest PROTO, pArray:PTR SDWORD, count:DWORD

.data
array1 SDWORD 10,-5,3,99,-100
array2 SDWORD -7,-3,-9
array3 SDWORD 1,5,5,5,5,2,9,-8
.code
main PROC
    INVOKE FindLargest, ADDR array1, LENGTHOF array1
    call WriteInt
    call Crlf
    INVOKE FindLargest, ADDR array2, LENGTHOF array2
    call WriteInt
    call Crlf
    INVOKE FindLargest, ADDR array3, LENGTHOF array3
    call WriteInt
    call Crlf
    exit
main ENDP

FindLargest PROC USES ecx esi, pArray:PTR SDWORD, count:DWORD
    mov  esi,pArray
    mov  ecx,count
    mov  eax,[esi]          ; 첫 요소를 최댓값으로 가정
    add  esi,4
    dec  ecx
    jz   Done               ; 요소가 1개면 끝
L1:
    cmp  eax,[esi]
    jge  Next               ; 부호 있는 비교
    mov  eax,[esi]
Next:
    add  esi,4
    loop L1
Done:
    ret
FindLargest ENDP
END main
```

```
+99
-3
+9
```

★ 2. 체스판

```
INCLUDE Irvine32.inc

BOARD_SIZE = 8
SQ_W = 6                    ; 사각형 너비(열)
SQ_H = 3                    ; 사각형 높이(행)

DrawBoard  PROTO, darkColor:DWORD
DrawRow    PROTO, row:DWORD, darkColor:DWORD
DrawSquare PROTO, row:DWORD, col:DWORD, color:DWORD
FillLine   PROTO, x:DWORD, y:DWORD

.code
main PROC
    call ClrScr
    INVOKE DrawBoard, gray
    mov  eax,lightGray + (black * 16)    ; 기본 색 복원
    call SetTextColor
    mov  dh,BOARD_SIZE * SQ_H
    mov  dl,0
    call Gotoxy
    exit
main ENDP

DrawBoard PROC USES ecx, darkColor:DWORD
    mov  ecx,0
L1:
    INVOKE DrawRow, ecx, darkColor
    inc  ecx
    cmp  ecx,BOARD_SIZE
    jb   L1
    ret
DrawBoard ENDP

DrawRow PROC USES eax ecx, row:DWORD, darkColor:DWORD
    mov  ecx,0
L1:
    mov  eax,row
    add  eax,ecx
    test eax,1
```

```
    jz   IsWhite            ; (행+열)이 짝수면 흰색
    mov  eax,darkColor
    jmp  Draw
IsWhite:
    mov  eax,white
Draw:
    INVOKE DrawSquare, row, ecx, eax
    inc  ecx
    cmp  ecx,BOARD_SIZE
    jb   L1
    ret
DrawRow ENDP

DrawSquare PROC USES eax ebx ecx edx, row:DWORD, col:DWORD, color:DWORD
    mov  eax,color
    shl  eax,4              ; 배경색 = color, 글자색 = 검정
    call SetTextColor
    imul ebx,col,SQ_W       ; 시작 X
    imul eax,row,SQ_H       ; 시작 Y
    mov  ecx,SQ_H
L1:
    INVOKE FillLine, ebx, eax
    inc  eax
    loop L1
    ret
DrawSquare ENDP

FillLine PROC USES eax ecx edx, x:DWORD, y:DWORD
    mov  dl,BYTE PTR x
    mov  dh,BYTE PTR y
    call Gotoxy
    mov  ecx,SQ_W
    mov  al,' '
L1:
    call WriteChar
    loop L1
    ret
FillLine ENDP
END main
```

★★★ 3. 색이 번갈아 바뀌는 체스판

프로시저(DrawBoard, DrawRow, DrawSquare, FillLine)와 상수·PROTO는 2번과 동일하고, main만 다음과 같이 바뀝니다.

```
main PROC
    call ClrScr
    mov  ebx,0                  ; 배경색 번호 0 ~ 15
    mov  ecx,16
L1:
    INVOKE DrawBoard, ebx       ; 흰색 칸은 그대로, 색 칸만 ebx 색
    mov  eax,500
    call Delay                  ; 500ms
    inc  ebx
    loop L1
    mov  eax,lightGray + (black * 16)
    call SetTextColor
    mov  dh,BOARD_SIZE * SQ_H
    mov  dl,0
    call Gotoxy
    exit
main ENDP
```

★★ 4. FindThrees

```
INCLUDE Irvine32.inc

FindThrees PROTO, pArray:PTR DWORD, arraySize:DWORD

.data
a1 DWORD 1,3,3,3,5
a2 DWORD 3,3,1,3,3
a3 DWORD 3,3
a4 DWORD 1,2,3,4,5,6,3,3,3
.code
main PROC
    INVOKE FindThrees, ADDR a1, LENGTHOF a1
    call WriteDec
    call Crlf
    INVOKE FindThrees, ADDR a2, LENGTHOF a2
    call WriteDec
    call Crlf
    INVOKE FindThrees, ADDR a3, LENGTHOF a3
    call WriteDec
    call Crlf
    INVOKE FindThrees, ADDR a4, LENGTHOF a4
    call WriteDec
    call Crlf
    exit
main ENDP

FindThrees PROC USES ecx esi, pArray:PTR DWORD, arraySize:DWORD
    mov  esi,pArray
```

```
    mov  ecx,arraySize
    sub  ecx,2              ; 시작 가능한 위치 수 = 크기 - 2
    jbe  NotFound           ; 크기가 2 이하
L1:
    cmp  DWORD PTR [esi],3
    jne  Next
    cmp  DWORD PTR [esi+4],3
    jne  Next
    cmp  DWORD PTR [esi+8],3
    je   Found
Next:
    add  esi,4
    loop L1
NotFound:
    mov  eax,0
    ret
Found:
    mov  eax,1
    ret
FindThrees ENDP
END main
```

```
1
0
0
1
```

★★ 5. DifferentInputs

```
INCLUDE Irvine32.inc

DifferentInputs PROTO, a:DWORD, b:DWORD, c:DWORD

.code
main PROC
    INVOKE DifferentInputs, 1, 2, 3
    call WriteDec
    call Crlf
    INVOKE DifferentInputs, 4, 4, 5
    call WriteDec
    call Crlf
    INVOKE DifferentInputs, 6, 7, 6
    call WriteDec
    call Crlf
    INVOKE DifferentInputs, 8, 9, 9
    call WriteDec
    call Crlf
    INVOKE DifferentInputs, 100, -100, 0
    call WriteDec
    call Crlf
    exit
main ENDP

DifferentInputs PROC, a:DWORD, b:DWORD, c:DWORD
    mov  eax,a
    cmp  eax,b
    je   Same               ; a = b
    cmp  eax,c
    je   Same               ; a = c
    mov  eax,b
    cmp  eax,c
    je   Same               ; b = c
    mov  eax,1
    ret
Same:
    mov  eax,0
    ret
DifferentInputs ENDP
END main
```

```
1
0
0
0
1
```

★★ 6. 정수 교환 (Swap)

```
INCLUDE Irvine32.inc

Swap PROTO, pValX:PTR DWORD, pValY:PTR DWORD

.data
array DWORD 7,3,9,1,8,2,6,4,5,0
.code
main PROC
    mov  esi,OFFSET array
    mov  ecx,(LENGTHOF array) / 2      ; 쌍의 개수
L1:
    lea  eax,[esi+4]
    INVOKE Swap, esi, eax              ; 연속된 두 원소 교환
```

```
    add  esi,8
    loop L1

    mov  esi,OFFSET array              ; 결과 확인
    mov  ecx,LENGTHOF array
    mov  ebx,TYPE array
    call DumpMem
    exit
main ENDP

Swap PROC USES eax esi edi, pValX:PTR DWORD, pValY:PTR DWORD
    mov  esi,pValX
    mov  edi,pValY
    mov  eax,[esi]
    xchg eax,[edi]
    mov  [esi],eax
    ret
Swap ENDP
END main
```

```
교환 후 배열: 3 7 1 9 2 8 4 6 0 5
```

★★ 7. 최대공약수 (재귀)

```
INCLUDE Irvine32.inc

GCD PROTO, x:DWORD, y:DWORD

.data
pairs  DWORD 5,20, 24,18, 11,7, 432,226, 26,13
strGcd BYTE "GCD = ",0
.code
main PROC
    mov  esi,OFFSET pairs
    mov  ecx,(LENGTHOF pairs) / 2
L1:
    INVOKE GCD, DWORD PTR [esi], DWORD PTR [esi+4]
    mov  edx,OFFSET strGcd
    call WriteString
    call WriteDec
    call Crlf
    add  esi,8
    loop L1
    exit
main ENDP

; GCD(x,y) = x            (y = 0)
;          = GCD(y, x mod y)
GCD PROC USES edx, x:DWORD, y:DWORD
    mov  eax,x
    cmp  y,0
    je   Done               ; 종료 조건: 결과는 EAX = x
    mov  edx,0
    div  y                  ; EDX = x mod y
    INVOKE GCD, y, edx      ; 재귀 호출
Done:
    ret
GCD ENDP
END main
```

```
GCD = 5
GCD = 6
GCD = 1
GCD = 2
GCD = 13
```

★★ 8. CountMatches

```
INCLUDE Irvine32.inc

CountMatches PROTO, pX:PTR SDWORD, pY:PTR SDWORD, count:DWORD

.data
x1 SDWORD 1,2,3,4,5
y1 SDWORD 1,0,3,0,5
x2 SDWORD -5,10,-15,20
y2 SDWORD 5,10,-15,-20
.code
main PROC
    INVOKE CountMatches, ADDR x1, ADDR y1, LENGTHOF x1
    call WriteDec
    call Crlf
    INVOKE CountMatches, ADDR x2, ADDR y2, LENGTHOF x2
    call WriteDec
    call Crlf
    exit
main ENDP

CountMatches PROC USES ebx ecx esi edi, pX:PTR SDWORD, pY:PTR SDWORD, count:DWORD
    mov  esi,pX
    mov  edi,pY
```

```
    mov  ecx,count
    mov  eax,0              ; 일치 개수
    jecxz Done
L1:
    mov  ebx,[esi]
    cmp  ebx,[edi]
    jne  Next
    inc  eax
Next:
    add  esi,4
    add  edi,4
    loop L1
Done:
    ret
CountMatches ENDP
END main
```

```
3
2
```

★★★ 9. CountNearMatches

```
INCLUDE Irvine32.inc

CountNearMatches PROTO, pX:PTR SDWORD, pY:PTR SDWORD, count:DWORD, diff:DWORD

.data
x1 SDWORD 10,20,30,40,50
y1 SDWORD 11,18,30,45,100
x2 SDWORD -10,-5,0,5
y2 SDWORD -12,-4,3,9
.code
main PROC
    INVOKE CountNearMatches, ADDR x1, ADDR y1, LENGTHOF x1, 2
    call WriteDec
    call Crlf
    INVOKE CountNearMatches, ADDR x2, ADDR y2, LENGTHOF x2, 3
    call WriteDec
    call Crlf
    exit
main ENDP

CountNearMatches PROC USES ebx ecx esi edi, pX:PTR SDWORD, pY:PTR SDWORD, count:DWORD, diff:DWORD
    mov  esi,pX
    mov  edi,pY
    mov  ecx,count
    mov  eax,0
    jecxz Done
L1:
    mov  ebx,[esi]
    sub  ebx,[edi]          ; x - y
    jns  Positive
    neg  ebx                ; 절댓값
Positive:
    cmp  ebx,diff
    ja   Next               ; |x - y| > diff 이면 건너뜀
    inc  eax
Next:
    add  esi,4
    add  edi,4
    loop L1
Done:
    ret
CountNearMatches ENDP
END main
```

```
3
3
```

|x − y|가 오버플로 없이 계산되는 값 범위(그리고 diff ≥ 0)라고 가정했습니다.

★★★ 10. ShowParams

```
INCLUDE Irvine32.inc

MySample   PROTO, first:DWORD, second:DWORD, third:DWORD
ShowParams PROTO, paramCount:DWORD

.data
titleStr BYTE "Stack parameters:",0dh,0ah
         BYTE "--------------------------",0dh,0ah,0
addrStr  BYTE "Address ",0
eqStr    BYTE " = ",0
.code
main PROC
    INVOKE MySample, 1234h, 5000h, 6543h
    exit
main ENDP

MySample PROC first:DWORD, second:DWORD, third:DWORD
    paramCount = 3
```

```
    INVOKE ShowParams, paramCount
    ret
MySample ENDP

ShowParams PROC USES eax ecx edx esi, paramCount:DWORD
    mov  edx,OFFSET titleStr
    call WriteString
    mov  esi,[ebp]          ; 호출자(MySample)의 EBP
    add  esi,8              ; EBP+8 = 첫 번째 매개변수 (가장 낮은 주소)
    mov  ecx,paramCount
L1:
    mov  edx,OFFSET addrStr
    call WriteString
    mov  eax,esi
    call WriteHex           ; 주소
    mov  edx,OFFSET eqStr
    call WriteString
    mov  eax,[esi]
    call WriteHex           ; 값
    call Crlf
    add  esi,4              ; 낮은 주소 → 높은 주소
    loop L1
    ret
ShowParams ENDP
END main
```

```
Stack parameters:
-------------------------
Address 0012FF80 = 00001234
Address 0012FF84 = 00005000
Address 0012FF88 = 00006543
(주소 값은 실행 환경에 따라 다름)
```